

A **bounded generic class** and a **final class** solve two completely different problems in Java:

- **Bounded generic class** → restricts **what types can be used as type arguments**.
- **Final class** → restricts **which classes can inherit from a class**.

They are not alternatives to each other, and a class can be **both bounded-generic and final**.

1. Bounded Generic Class

Definition

A **generic class** is a class that works with a type parameter.

A **bounded generic class** is a generic class in which the type parameter is restricted to a particular class/interface hierarchy using a bound.

The basic syntax is:

```
class ClassName<T extends SomeClass> {  
    // ...  
}
```

Here:

- T = type parameter
- extends = upper-bound operator for a type parameter
- SomeClass = upper bound
- T can represent SomeClass or one of its subclasses.

Example

```
class NumberBox<T extends Number> {  
  
    private T value;  
  
    NumberBox(T value) {  
        this.value = value;  
    }  
  
    T getValue() {  
        return value;  
    }  
  
    double getDoubleValue() {  
        return value.doubleValue();  
    }  
}
```

Now:

```
NumberBox<Integer> a = new NumberBox<>(100);
NumberBox<Double> b = new NumberBox<>(25.5);
NumberBox<Long> c = new NumberBox<>(1000L);
```

These are valid because:

```
Integer
Double
Long
↓
Number
```

But:

```
NumberBox<String> x = new NumberBox<>("Hello");
```

is invalid because String is not a subclass of Number.

2. Why Use a Bound?

The bound gives the generic class a guarantee about the operations available on T.

For example:

```
class Calculator<T extends Number> {
    T value;

    Calculator(T value) {
        this.value = value;
    }

    double calculate() {
        return value.doubleValue();
    }
}
```

Because T extends Number, Java knows that T has access to methods inherited from Number, such as:

```
intValue()
longValue()
floatValue()
doubleValue()
```

Without the bound:

```
class Calculator<T> {  
    T value;  
  
    double calculate() {  
        return value.doubleValue(); // ERROR  
    }  
}
```

Java cannot assume that arbitrary T has a doubleValue() method.

3. Types of Bounds

Java primarily has:

Upper bound

<T extends Number>

Meaning:

T must be Number or a subclass of Number

Multiple bounds

<T extends Number & Comparable<T>>

Meaning T must satisfy both constraints.

Example:

```
class Processor<T extends Number & Comparable<T>> {  
    T value;  
  
    Processor(T value) {  
        this.value = value;  
    }  
  
    int compare(T other) {  
        return value.compareTo(other);  
    }  
}
```

The first bound can be a class, followed by interfaces.

4. Bounded Generic Class Does NOT Prevent Inheritance

This is important.

Consider:

```
class Box<T extends Number> {  
}
```

You can still extend Box:

```
class IntegerBox extends Box<Integer> {  
}
```

You can also extend it with another generic class:

```
class SpecialBox<T extends Number> extends Box<T> {  
}
```

Therefore:

```
bounded generic  
  ↓  
restricts TYPE ARGUMENTS
```

It does **not** mean:

cannot be inherited

5. Final Class

Definition

A final class is a class that **cannot be extended**.

Syntax:

```
final class ClassName {  
}
```

Example:

```
final class Account {
```

```
void display() {  
    System.out.println("Account");  
}  
}
```

This is valid:

```
Account a = new Account();  
a.display();
```

But this is invalid:

```
class SavingsAccount extends Account {  
}
```

Compiler error:

cannot inherit from final Account

6. Why Use a Final Class?

A class is commonly declared final when its implementation should not be changed through inheritance.

For example:

```
final class SecurityConfiguration {  
  
    void configure() {  
        System.out.println("Secure configuration");  
    }  
}
```

Nobody can create:

```
class UnsafeConfiguration extends SecurityConfiguration {  
  
    @Override  
    void configure() {  
        System.out.println("Modified configuration");  
    }  
}
```

because inheritance itself is prohibited.

A well-known Java example is:

String

String is a final class.

Conceptually:

String

↑

cannot be extended

7. Main Difference

Parameter	Bounded Generic Class	Final Class
Main purpose	Restricts type arguments	Prevents inheritance
Related concept	Generics	Inheritance
Syntax	class Box<T extends Number>	final class Box
Restricts T?	Yes	No
Prevents subclassing?	No	Yes
Works with type parameters?	Yes	Not necessarily
Can have subclasses?	Yes, unless separately declared final	No
Can be instantiated?	Yes, subject to normal rules	Yes, subject to normal rules
Controls compile-time type compatibility?	Yes	Yes, but through inheritance restriction
Controls method overriding through inheritance?	No	Yes, because subclasses cannot exist
Can use interfaces as bounds?	Yes	Not applicable
Can have multiple bounds?	Yes	Not applicable
Can be both?	Yes	Yes

8. Very Important: extends Has Two Different Meanings

This is one of the most confusing aspects of Java.

In a class declaration

```
class Dog extends Animal {  
}
```

Here:

extends

means:

Dog inherits from Animal.

In a generic type parameter

```
class Box<T extends Number> {  
}
```

Here:

extends

means:

T is restricted to Number or its subclasses.

It does **not** mean that T itself is a class declaration.

Compare:

```
class Dog extends Animal  
  ↓  
inheritance relationship
```

```
<T extends Number>  
  ↓  
generic type bound
```

9. A Class Can Be Both Bounded Generic and Final

This is perfectly legal:

```
final class NumberBox<T extends Number> {  
  
    private T value;  
  
    NumberBox(T value) {  
        this.value = value;  
    }  
  
    T getValue() {  
        return value;  
    }  
}
```

Now two independent restrictions exist.

Restriction 1 — Generic bound

<T extends Number>

Only Number subclasses can be used:

```
NumberBox<Integer> a = new NumberBox<>(10);  
NumberBox<Double> b = new NumberBox<>(20.5);
```

But:

```
NumberBox<String> c = new NumberBox<>("Hello");
```

is invalid.

Restriction 2 — final

```
final class NumberBox
```

Nobody can do:

```
class MyBox extends NumberBox<Integer> {  
}
```

That is invalid because NumberBox is final.

So:

```
final class NumberBox<T extends Number>
    |
    |_____ restricts T
    |
    |_____ prevents subclassing
```

10. Bounded Generic Class vs Final Generic Class

These two are especially easy to confuse.

Bounded generic

```
class Box<T extends Number> {
}
```

Restriction:

$T \rightarrow \text{Number or subclass}$

Inheritance is still possible:

```
class SpecialBox extends Box<Integer> {
}
```

Final generic

```
final class Box<T> {
}
```

Restriction:

$\text{Box} \rightarrow \text{cannot be extended}$

But the type parameter has no bound.

Therefore:

```
Box<Integer> a = new Box<>();
Box<String> b = new Box<>();
Box<Employee> c = new Box<>();
```

could all be valid.

11. Four Important Combinations

Understanding these four combinations makes the distinction very clear.

Case 1 — Generic but not bounded

```
class Box<T> {  
}
```

Meaning:

Any reference type can normally be supplied as T
+
Box can be extended

Example:

```
class SpecialBox extends Box<String> {  
}
```

Case 2 — Bounded generic but not final

```
class NumberBox<T extends Number> {  
}
```

Meaning:

T must be Number/subclass
+
NumberBox can be extended

Example:

```
class IntegerBox extends NumberBox<Integer> {  
}
```

Case 3 — Final generic but unbounded

```
final class Box<T> {  
}
```

Meaning:

T has no declared upper bound
+
Box cannot be extended

Case 4 — Final bounded generic

```
final class NumberBox<T extends Number> {  
}
```

Meaning:

T must be Number/subclass

+

NumberBox cannot be extended

This combines both restrictions.

12. Example With a Real Hierarchy

Suppose:

```
class Animal {  
}
```

```
class Dog extends Animal {  
}
```

```
class Cat extends Animal {  
}
```

```
class Vehicle {  
}
```

Now:

```
class AnimalBox<T extends Animal> {  
  
    private T animal;  
  
    AnimalBox(T animal) {  
        this.animal = animal;  
    }  
  
    T getAnimal() {  
        return animal;  
    }  
}
```

Valid:

```
AnimalBox<Animal> a = new AnimalBox<>{new Animal()};
```

```
AnimalBox<Dog> d = new AnimalBox<>{new Dog()};
```

```
AnimalBox<Cat> c = new AnimalBox<>{new Cat()};
```

Invalid:

```
AnimalBox<Vehicle> v = new AnimalBox<>{new Vehicle()};
```

because:

```
Vehicle
  X
  |
AnimalBox<T extends Animal>
```

But inheritance is still allowed:

```
class DogBox extends AnimalBox<Dog> {
    DogBox(Dog dog) {
        super(dog);
    }
}
```

Now make the generic class final:

```
final class AnimalBox<T extends Animal> {
    private T animal;

    AnimalBox(T animal) {
        this.animal = animal;
    }
}
```

Then:

```
class DogBox extends AnimalBox<Dog> {
}
```

becomes invalid.

13. The Conceptual Difference

Think of the two mechanisms as controlling **different levels**.

Bounded generic

Controls:

WHAT TYPE goes into the class?

Example:

Box<T extends Number>

asks:

Is T a valid subtype of Number?

Final class

Controls:

WHO CAN EXTEND the class?

Example:

final class Box<T>

asks:

Is someone trying to create a subclass of Box?

14. Bounded Generic ≠ Final Type

Another important distinction:

<T extends Number>

does **not** mean:

T is final

It means:

T belongs somewhere under Number in the inheritance hierarchy.

For example:

```
class NumberBox<T extends Number> {  
}
```

can accept:

Integer
Double
Long
Float
Short
Byte
BigDecimal
BigInteger

subject to the actual hierarchy and type compatibility.

The bound doesn't make those types final.

15. Final Type ≠ Bounded Generic

Likewise:

```
final class Employee {  
}
```

doesn't make it a generic class.

It simply means:

Employee cannot be a superclass.

You can still use it as a generic argument:

```
class Box<T> {  
}
```

```
Box<Employee> box = new Box<>();
```

So a final class can be used as T.

16. Relationship With final Type Parameters

Java also allows a final variable/reference:

```
final NumberBox<Integer> box = new NumberBox<>{10};
```

This means the **reference cannot be reassigned**:

```
box = new NumberBox<>{20}; // ERROR
```

But this has nothing to do with:

```
final class NumberBox
```

These are three different concepts:

Syntax	Meaning
final class A	A cannot be subclassed
final int x	x cannot be reassigned
<T extends Number>	T must satisfy a generic bound

17. Bounded Generic vs Final vs Sealed

Java also provides sealed classes, which are useful for understanding the distinction.

Final

```
final class Animal {  
}
```

Inheritance:

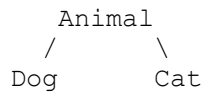
```
Animal  
X  
├── Dog  
└── Cat
```

No subclasses.

Sealed

```
sealed class Animal permits Dog, Cat {  
}
```

Only specified classes can extend it:



Bounded generic

```
class Box<T extends Animal> {  
}
```

This doesn't restrict who subclasses AnimalBox or Animal.

It restricts the **type argument**:

Box<Animal>	✓
Box<Dog>	✓
Box<Cat>	✓
Box<Car>	✗

18. A Simple Memory Trick

Remember:

Bound = What can T be?

<T extends Number>

means:

"What types are allowed for T?"

And:

Final = Who can extend this class?

final class NumberBox

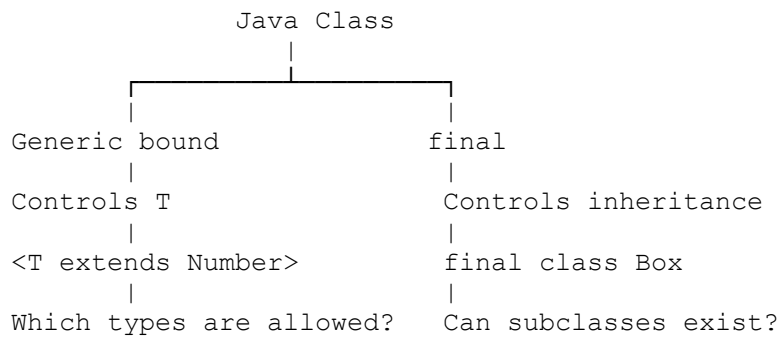
means:

"Can another class extend NumberBox?"

Answer:

No.

19. Final Summary



In one sentence:

A bounded generic class restricts the types that can be supplied to its type parameter, whereas a final class prevents other classes from inheriting from it.

And they can be combined:

```
final class NumberBox<T extends Number> {
}
```

Here:

T extends Number



restricts the TYPE ARGUMENT

final



restricts INHERITANCE

So the most important distinction is:

Bounded generics control type arguments; final controls inheritance.

In Java, **multiple bounds** are a feature of **bounded type parameters** in generics. They allow a type parameter to be constrained by **more than one type**—typically **one class plus one or more interfaces**, or multiple interfaces.

A useful distinction is:

- **Single-bounded generic:** T extends Number
 - **Multiple-bounded generic:** T extends Number & Comparable<T> & Serializable
-

1. What is a bounded generic class?

A bounded generic class is a generic class whose type parameter has a restriction.

Unbounded generic class

```
class Box<T> {  
    private T value;  
  
    Box(T value) {  
        this.value = value;  
    }  
}
```

Here T has no explicit upper bound.

You could use:

```
Box<String> b1 = new Box<>("Hello");  
Box<Integer> b2 = new Box<>(100);  
Box<Employee> b3 = new Box<>(new Employee());
```

Single-bounded generic class

```
class NumberBox<T extends Number> {  
    private T value;  
  
    NumberBox(T value) {  
        this.value = value;  
    }  
  
    double getDoubleValue() {  
        return value.doubleValue();  
    }  
}
```

The declaration:

`<T extends Number>`

means:

T must be Number or a subtype of Number.

For example:

```
NumberBox<Integer> a = new NumberBox<>(100);
NumberBox<Double> b = new NumberBox<>(25.5);
NumberBox<Long> c = new NumberBox<>(1000L);
```

But:

```
NumberBox<String> x = new NumberBox<>("Hello");
```

is invalid.

2. What is a multiple-bounded generic class?

A **multiple-bounded generic class** is a generic class whose type parameter has **multiple bounds**.

The syntax is:

```
class ClassName<T extends ClassName & Interface1 & Interface2> {
}
```

For example:

```
class Processor<T extends Number & Comparable<T>> {

    private T value;

    Processor(T value) {
        this.value = value;
    }

    double getDoubleValue() {
        return value.doubleValue();
    }

    int compareWith(T other) {
        return value.compareTo(other);
    }
}
```

}

Here T has two bounds:

T extends Number

&

T implements Comparable<T>

Therefore, a valid T must satisfy **both** conditions.

3. Understanding the & operator

In a multiple bound:

<T extends Number & Comparable<T>>

the & means:

The type argument must satisfy **all** the specified bounds.

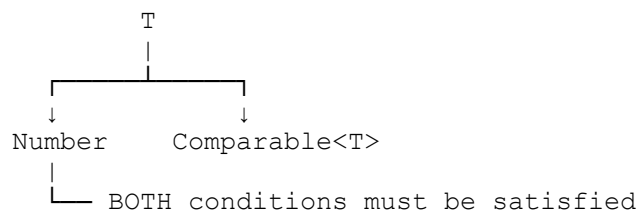
It is not saying:

Number OR Comparable

It means:

Number AND Comparable<T>

Conceptually:



4. Why would we need multiple bounds?

Suppose you want a generic class to work with numbers.

You want to use:

doubleValue()

That comes from Number.

But you also want to compare two values:

`compareTo()`

That comes from Comparable.

A single bound:

`<T extends Number>`

gives you:

`doubleValue()`

but it does not guarantee that T implements `Comparable<T>`.

A multiple bound:

`<T extends Number & Comparable<T>>`

gives you both sets of capabilities.

Therefore:

```
class Processor<T extends Number & Comparable<T>> {  
  
    double convert(T value) {  
        return value.doubleValue();  
    }  
  
    int compare(T a, T b) {  
        return a.compareTo(b);  
    }  
}
```

Java can safely compile both operations.

5. Single Bound vs Multiple Bounds

Feature	Single bounded generic	Multiple-bounded generic
Number of bounds	One	Two or more
Example	<code><T extends Number></code>	<code><T extends Number &</code>

		Comparable<T>>
Main purpose	One restriction	Multiple simultaneous restrictions
Logical relationship	One requirement	AND relationship
Can have a class bound?	Yes	Yes
Can have interface bounds?	Yes	Yes
Can have multiple interfaces?	No	Yes
Can have multiple classes?	No	No
Syntax	T extends A	T extends A & B & C
First bound	Can be class/interface	If a class is present, it must be first
Access to members	Members guaranteed by bound	Members guaranteed by all bounds
Flexibility	More flexible	More restrictive
Type must satisfy	One condition	Every condition

6. The Most Important Rule: Only One Class Bound

Java does **not** allow:

<T extends Number & Integer>

because both Number and Integer are classes.

Likewise:

<T extends Animal & Vehicle>

is illegal if both Animal and Vehicle are classes.

Java allows:

One class + multiple interfaces

or:

Multiple interfaces

For example:

`<T extends Number & Comparable<T> & Serializable>`

is valid.

The structure is:

```
<T extends
    Class
    &
    Interface
    &
    Interface
    &
    Interface>
```

7. The Class Must Come First

This is valid:

`<T extends Number & Comparable<T> & Serializable>`

This is invalid:

`<T extends Comparable<T> & Number>`

when Number is the class bound.

The class must appear before the interfaces.

Correct:

`<T extends Number & Interface1 & Interface2>`

Incorrect:

`<T extends Interface1 & Number & Interface2>`

8. Example: Number + Comparable

Consider:

```

class NumericProcessor<T extends Number & Comparable<T>> {

    public double convert(T value) {
        return value.doubleValue();
    }

    public int compare(T a, T b) {
        return a.compareTo(b);
    }
}

```

Now:

```

NumericProcessor<Integer> processor =
    new NumericProcessor<>();

System.out.println(processor.convert(100));

System.out.println(processor.compare(10, 20));

```

Why is Integer valid?

Because:

```

Integer
|
├── extends Number
└── implements Comparable<Integer>

```

Therefore:

Integer satisfies Number
AND
Integer satisfies Comparable<Integer>

9. What Happens With an Invalid Type?

Suppose:

```

class Employee {
}

```

and:

```

NumericProcessor<Employee> processor =
    new NumericProcessor<>();

```


This is invalid because Employee does not satisfy:

T extends Number

Even if you changed Employee to:

```
class Employee implements Comparable<Employee> {  
  
    @Override  
    public int compareTo(Employee other) {  
        return 0;  
    }  
}
```

it would still be invalid.

Why?

Because now:

Employee
✓ Comparable<Employee>
✗ Number

A multiple-bound type must satisfy **every bound**.

10. Multiple Interface Bounds

Multiple bounds don't necessarily require a class bound.

For example:

```
interface Printable {  
    void print();  
}
```

```
interface Loggable {  
    void log();  
}
```

```
class Utility<T extends Printable & Loggable> {  
  
    void process(T value) {  
        value.print();  
        value.log();  
    }  
}
```

```
}
```

Here:

<T extends Printable & Loggable>

means:

T must implement Printable

AND

T must implement Loggable

A valid class:

```
class Report implements Printable, Loggable {
```

```
    @Override
    public void print() {
        System.out.println("Printing report");
    }
```

```
    @Override
    public void log() {
        System.out.println("Logging report");
    }
}
```

Now:

```
Utility<Report> utility = new Utility<>();
utility.process(new Report());
```

works.

11. Why Multiple Bounds Are Powerful

Without multiple bounds:

```
class Utility<T> {

    void process(T value) {
        value.print(); // ERROR
        value.log();  // ERROR
    }
}
```

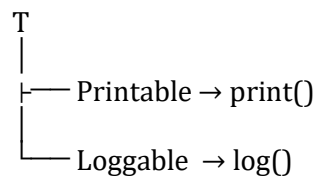
Java knows nothing about T.

With:

```
class Utility<T extends Printable & Loggable> {  
  
    void process(T value) {  
        value.print();  
        value.log();  
    }  
}
```

Java knows that every valid T must provide both methods.

Therefore the compiler can guarantee:



12. Three Bounds

You can have more than two bounds.

For example:

```
interface Printable {  
    void print();  
}
```

```
interface Loggable {  
    void log();  
}
```

```
interface Validatable {  
    boolean validate();  
}
```

Then:

```
class Processor<  
    T extends Printable  
      & Loggable  
      & Validatable> {  
  
    void process(T value) {  
  
        if (value.validate()) {
```

```

        value.print();
        value.log();
    }
}

```

Now T must implement:

```

Printable
+
Loggable
+
Validatable

```

A class satisfying all three:

```

class Document
    implements Printable, Loggable, Validatable {

    public void print() {
        System.out.println("Printing");
    }

    public void log() {
        System.out.println("Logging");
    }

    public boolean validate() {
        return true;
    }
}

```

Then:

```

Processor<Document> processor = new Processor<>();
processor.process(new Document());

```

is valid.

13. Four Bounds

You can continue:

```
<T extends A & B & C & D>
```

provided the bounds obey Java's rules.

For example:

```
class Processor<
    T extends Number
      & Comparable<T>
      & Serializable
      & SomeInterface> {
}
```

This means:

T must be:

```
Number
AND
Comparable<T>
AND
Serializable
AND
SomeInterface
```

14. Single Bound Example

Consider:

```
class AnimalProcessor<T extends Animal> {

    void process(T animal) {
        animal.eat();
    }
}
```

If:

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}
```

```
class Dog extends Animal {
}
```

```
class Cat extends Animal {
}
```

then:

```
AnimalProcessor<Dog> dogProcessor =  
    new AnimalProcessor<>();
```

```
AnimalProcessor<Cat> catProcessor =  
    new AnimalProcessor<>();
```

Both are valid.

The only requirement is:

T must be an Animal

15. Multiple Bound Example

Now suppose you want an object to be both an animal and identifiable.

```
interface Identifiable {  
    int getId();  
}
```

You can write:

```
class AnimalProcessor<  
    T extends Animal & Identifiable> {  
  
    void process(T animal) {  
        animal.eat();  
        System.out.println(animal.getId());  
    }  
}
```

Now the requirements are:

T must be Animal

AND

T must implement Identifiable

A valid class:

```
class Dog extends Animal implements Identifiable {  
  
    @Override  
    public int getId() {  
        return 101;  
    }  
}
```

Then:

```
AnimalProcessor<Dog> processor =  
    new AnimalProcessor<>();
```

is valid.

16. Single Bound Is Less Restrictive

Consider:

<T extends Animal>

Valid:

Animal
Dog
Cat
Horse
Elephant
...

provided they are subclasses of Animal.

Now:

<T extends Animal & Identifiable>

requires:

Animal
AND
Identifiable

Therefore:

Animal	
Dog	✓ if Identifiable
Cat	✓ if Identifiable
Horse	✓ if Identifiable

A subclass of Animal that doesn't implement Identifiable is rejected.

Thus:

Adding bounds reduces the set of legal type arguments.

17. Comparison Through Set Theory

You can think of it this way.

For a single bound:

<T extends Animal>

the allowed set is:

Animal family

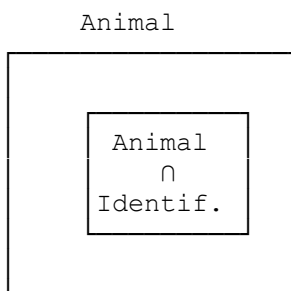
For:

<T extends Animal & Identifiable>

the allowed set is the intersection:

$\text{Animal} \cap \text{Identifiable}$

Conceptually:



The multiple-bound set is therefore **smaller or equal** to the single-bound set.

18. Bounds and Available Methods

This is one of the biggest practical reasons for multiple bounds.

Single bound

<T extends Number>

Java guarantees:


```
value.intValue();
value.longValue();
value.doubleValue();
```

because these come from Number.

But Java cannot automatically assume:

```
value.compareTo(other);
```

unless the bound also guarantees Comparable.

Multiple bounds

<T extends Number & Comparable<T>>

Java guarantees both:

```
value.doubleValue();
```

and:

```
value.compareTo(other);
```

Therefore:

```
class Processor<T extends Number & Comparable<T>> {
    void process(T a, T b) {
        double x = a.doubleValue();
        int result = a.compareTo(b);
        System.out.println(x);
        System.out.println(result);
    }
}
```

19. Multiple Bounds vs Multiple Type Parameters

These are **not the same**.

Multiple bounds

```
class Processor<T extends Number & Comparable<T>> {  
}
```

There is **one type parameter**:

T

but it has multiple constraints:

T → Number

T → Comparable<T>

Multiple type parameters

```
class Pair<T, U> {  
}
```

There are **two independent type parameters**:

T

U

For example:

Pair<Integer, String>

Here:

T = Integer

U = String

This has nothing to do with multiple bounds.

20. Multiple Bounds vs Wildcard Bounds

Another important distinction:

Type-parameter bound

```
class Box<T extends Number> {  
}
```

This defines a restriction on the type parameter T.

Wildcard bound

List<? extends Number>

This defines an upper bound on a wildcard.

Lower wildcard bound

List<? super Integer>

This is a lower bound.

So:

<T extends Number>

and:

? extends Number

are related concepts but are used in different generic constructs.

21. Multiple Bounds With Generic Methods

Multiple bounds can also be used on methods.

```
public static <T extends Number & Comparable<T>>
int compare(T a, T b) {

    return a.compareTo(b);
}
```

Usage:

```
int result = compare(10, 20);
```

Here Java infers:

T = Integer

and Integer satisfies both bounds.

22. Multiple Bounds With Generic Interfaces

You can also have:

```
interface Processor<T extends Number & Comparable<T>> {  
    void process(T value);  
}
```

Implementation:

```
class IntegerProcessor  
    implements Processor<Integer> {  
  
    @Override  
    public void process(Integer value) {  
        System.out.println(value);  
    }  
}
```

23. Important Restriction: Type Erasure

Java generics use **type erasure**.

With:

```
<T extends Number & Comparable<T>>
```

the **leftmost bound** determines the erasure of T.

For:

```
<T extends Number & Comparable<T>>
```

the erasure is effectively:

Number

This is another reason Java requires a class bound, if present, to appear first.

For example:

```
<T extends Number & Comparable<T> & Serializable>
```

has:

leftmost bound = Number

as the class bound.

24. Generic Class vs Multiple-Bounded Generic Class

Let's put the entire progression together.

Unbounded

```
class Box<T> {  
}
```

Requirement:

No explicit bound

Single bounded

```
class Box<T extends Number> {  
}
```

Requirement:

T must satisfy Number

Multiple bounded

```
class Box<T extends Number & Comparable<T>> {  
}
```

Requirements:

T must satisfy Number
AND
T must satisfy Comparable<T>

Multiple interfaces

```
class Box<T extends Printable & Loggable & Validatable> {  
}
```

Requirements:

T must satisfy Printable
AND
T must satisfy Loggable
AND
T must satisfy Validatable

25. Final Comparison

Concept	Example	Meaning
Unbounded generic	<code><T></code>	T has no explicit upper bound
Single class bound	<code><T extends Number></code>	T must be Number/subclass
Single interface bound	<code><T extends Comparable<T>></code>	T must implement Comparable
Multiple bounds	<code><T extends Number & Comparable<T>></code>	T must satisfy both
Multiple interface bounds	<code><T extends A & B & C></code>	T must implement all interfaces
Multiple type parameters	<code><T, U></code>	Two separate type parameters
Upper wildcard	<code>? extends Number</code>	Unknown type is Number/subtype
Lower wildcard	<code>? super Integer</code>	Unknown type is Integer/supertype

The key idea

Remember the following hierarchy:

`<T>`
└ No explicit bound

`<T extends A>`
└ Single bound

`<T extends A & B>`
└ Multiple bounds

`<T extends A & B & C>`
└ Multiple bounds

Where:

A = class or interface

B = interface

C = interface

when a class bound is present.

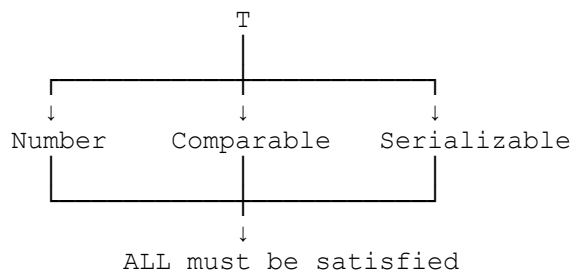
In one sentence:

A single-bounded generic class restricts its type parameter using one bound, while a multiple-bounded generic class restricts the same type parameter using multiple simultaneous bounds, and the supplied type must satisfy all of them.

The most important example to remember is:

```
class Processor<T extends Number & Comparable<T> & Serializable> {  
}
```

which means:



So **single bound = one constraint**, while **multiple bounds = AND-combination of constraints**.

The method

```
public static <T extends Comparable<T>> T max(T a, T b) {  
}
```

is a **generic static method with a bounded type parameter**. It says:

“Accept two values of the same type T, provided that T can be compared with another T, and return a value of type T.”

Let's break every part down.

1. Complete structure

```
public static <T extends Comparable<T>> T max(T a, T b) {  
    // method body  
}
```

There are **two different T concepts** in this declaration:

```
<T extends Comparable<T>>  
└── T used as Comparable's type argument  
└── declares the method's type parameter T
```

And then:

T max(T a, T b)

uses that declared type parameter.

So the overall structure is:

```
public  
↓  
access modifier  
  
static  
↓  
method belongs to class  
  
<T extends Comparable<T>>  
↓  
generic type parameter + upper bound  
  
T
```


↓
return type

max
↓
method name

(T a, T b)
↓
two parameters of type T

2. What does `<T extends Comparable<T>>` mean?

This is the most important part.

`<T extends Comparable<T>>`

means:

T must be a type that implements `Comparable<T>`.

In other words, the type represented by T must know how to compare itself with another object of the **same type**.

For example, Integer implements:

`Comparable<Integer>`

String implements:

`Comparable<String>`

Therefore both can be used as T.

Conceptually:

Integer
└─ implements Comparable<Integer>

String
└─ implements Comparable<String>

3. What is `Comparable<T>`?

Comparable<T> is a generic Java interface.

Conceptually, it contains:

```
public interface Comparable<T> {  
    int compareTo(T other);  
}
```

The important method is:

```
int compareTo(T other)
```

It compares the current object with other.

The result is conventionally:

Result	Meaning
< 0	Current object is smaller
0	Objects are considered equal in ordering
> 0	Current object is larger

For example:

```
Integer a = 10;
```

```
Integer b = 20;
```

```
System.out.println(a.compareTo(b));
```

The result is negative because:

```
10 < 20
```

4. Why is the bound necessary?

Suppose you wrote:

```
public static <T> T max(T a, T b) {  
    if (a.compareTo(b) > 0) {  
        return a;  
    }  
  
    return b;  
}
```

This doesn't compile.

Why?

Because:

`<T>`

only says:

T can be some reference type.

Java cannot assume that arbitrary T has a:

`compareTo()`

method.

For example:

```
class Employee {  
}
```

An Employee does not automatically have `compareTo()`.

5. The bound solves the problem

Now write:

```
public static <T extends Comparable<T>> T max(T a, T b) {  
    if (a.compareTo(b) > 0) {  
        return a;  
    }  
    return b;  
}
```

Now Java knows:

```
T  
↓  
implements Comparable<T>  
↓  
therefore has compareTo(T)
```

So:

`a.compareTo(b)`

is legal.

6. Complete working example

```
public class Demo {  
  
    public static <T extends Comparable<T>> T max(T a, T b) {  
  
        if (a.compareTo(b) > 0) {  
            return a;  
        }  
  
        return b;  
    }  
  
    public static void main(String[] args) {  
  
        Integer result1 = max(10, 20);  
  
        String result2 = max("Apple", "Banana");  
  
        System.out.println(result1);  
        System.out.println(result2);  
    }  
}
```

Output:

20

Banana

Why?

For the first call:

`max(10, 20)`

Java infers:

`T = Integer`

Therefore the method effectively becomes:

Integer max(Integer a, Integer b)

Conceptually.

For the second:

max("Apple", "Banana")

Java infers:

T = String

Conceptually:

String max(String a, String b)

7. Breaking down T step by step

Consider:

<T extends Comparable<T>>

First T

<T

declares a type parameter named T.

extends

<T extends

introduces an **upper bound**.

Comparable

<T extends Comparable

says the type must implement/extend Comparable.

Second T

<T extends Comparable<T>>

specifies the type that Comparable works with.

Therefore:

T must be Comparable TO T

or:

T must implement Comparable<T>.

8. Why isn't it simply Comparable?

You might wonder why we don't write:

<T extends Comparable>

That is a **raw type** and loses generic type safety.

The preferred form is:

<T extends Comparable<T>>

because it expresses:

T compares with another T

For example:

Integer implements Comparable<Integer>

not:

Integer implements Comparable<Something>

9. Why does the method return T?

Look at:

```
public static <T extends Comparable<T>> T max(T a, T b)
```

There are three occurrences of T:

T
↓
<T extends Comparable<T>> T max(T a, T b)

```
parameter  
parameter  
return type
```

The method:

takes T
takes T
returns T

So:

Integer x = max(10, 20);

returns an Integer.

And:

String x = max("A", "B");

returns a String.

10. Type inference

Java can usually determine T automatically.

For:

Integer result = max(10, 20);

Java sees:

a = Integer
b = Integer

and infers:

T = Integer

Then verifies:

Integer implements Comparable<Integer>

Yes.

Therefore the call is valid.

11. Explicitly specifying T

You can also explicitly provide the type argument:

```
Integer result =  
    Demo.<Integer>max(10, 20);
```

Here:

<Integer>

explicitly specifies:

T = Integer

For static generic methods, the syntax is:

ClassName.<Type>method(...)

For example:

```
Demo.<String>max("Apple", "Banana");
```

12. Example with String

```
public static <T extends Comparable<T>> T max(T a, T b) {  
    return a.compareTo(b) > 0 ? a : b;  
}
```

Call:

```
String result = max("Java", "Python");
```

Java infers:

T = String

Since:

String implements Comparable<String>

the call is valid.

Comparison is lexicographical.

Conceptually:

"Java"
vs
"Python"

"Python" is greater

Therefore:

Python

is returned.

13. Example with Integer

Integer result = max(100, 50);

Inference:

T = Integer

Comparison:

100.compareTo(50)

produces a positive value.

Therefore:

100

is returned.

14. Example with Double

Double result = max(10.5, 20.5);

Inference:

T = Double

And:

Double implements Comparable<Double>

Therefore:

20.5

is returned.

15. Creating your own comparable class

This is where the concept becomes especially useful.

Suppose:

```
class Employee implements Comparable<Employee> {  
  
    private int salary;  
  
    Employee(int salary) {  
        this.salary = salary;  
    }  
  
    @Override  
    public int compareTo(Employee other) {  
        return Integer.compare(this.salary, other.salary);  
    }  
  
    @Override  
    public String toString() {  
        return "Employee[salary=" + salary + "]";  
    }  
}
```

Now:

```
Employee e1 = new Employee(50000);  
Employee e2 = new Employee(80000);
```

You can use:

```
Employee higherPaid = max(e1, e2);
```

Why?

Because:

```
Employee  
↓  
implements Comparable<Employee>
```

Therefore:

Employee satisfies T extends Comparable<T>

The result is:

Employee[salary=80000]

16. What if a class doesn't implement Comparable?

Suppose:

```
class Employee {  
    private int salary;  
  
    Employee(int salary) {  
        this.salary = salary;  
    }  
}
```

Then:

```
Employee e1 = new Employee(50000);  
Employee e2 = new Employee(80000);
```

```
Employee result = max(e1, e2);
```

doesn't compile.

Why?

Because:

```
Employee  
    ✕  
Comparable<Employee>
```

Therefore:

T extends Comparable<T>

cannot be satisfied.

This is precisely what the generic bound is enforcing.

17. The method without generics

You could write a method specifically for integers:

```
public static Integer max(Integer a, Integer b) {  
    return a.compareTo(b) > 0 ? a : b;  
}
```

But now it only works for:

Integer

You would need separate methods for:

Integer
Long
Double
String
Employee
...

Generics eliminate that duplication:

```
public static <T extends Comparable<T>> T max(T a, T b)
```

One method can work with many comparable types.

18. What the generic method gives you

This single method:

```
public static <T extends Comparable<T>> T max(T a, T b)
```

can conceptually produce:

Integer max(Integer, Integer)
Long max(Long, Long)
Double max(Double, Double)
String max(String, String)
Employee max(Employee, Employee)
...

provided each type satisfies:

Comparable<T>

19. Why Comparable<T> instead of Comparable<Object>?

Consider:

<T extends Comparable<T>>

This means:

T.compareTo(T)

The comparison is between the same type.

For example:

Integer → Comparable<Integer>

so:

Integer.compareTo(Integer)

is valid.

This gives strong compile-time type checking.

20. Important distinction: Comparable<T> vs Comparator<T>

These are often confused.

Comparable

```
class Employee implements Comparable<Employee> {  
    @Override  
    public int compareTo(Employee other) {  
        ...  
    }  
}
```

The object defines its **natural ordering**.

Comparator

```
Comparator<Employee> comparator =  
    (e1, e2) -> Integer.compare{
```

```
        e1.salary,  
        e2.salary  
    );
```

The comparison logic is provided **externally**.

Your method:

```
<T extends Comparable<T>>
```

requires the type itself to provide the comparison through Comparable.

21. The limitation of <T extends Comparable<T>>

There is an important subtlety.

This bound requires an **exact self-type relationship**:

```
T implements Comparable<T>
```

Some types can have a slightly different form of comparability.

For example, a type might implement:

```
Comparable<SomeSuperType>
```

rather than:

```
Comparable<ThatExactType>
```

Such a type may not satisfy:

```
<T extends Comparable<T>>
```

even though it is technically comparable.

A more flexible version can sometimes use:

```
<T extends Comparable<? super T>>
```

For example:

```
public static <T extends Comparable<? super T>>  
T max(T a, T b) {  
  
    return a.compareTo(b) > 0 ? a : b;
```

}

This is a common pattern in Java library APIs because it accommodates comparison against T or a supertype of T.

22. Compare the two bounds

Strict version

`<T extends Comparable<T>>`

Means:

T must implement `Comparable<T>`

More flexible version

`<T extends Comparable<? super T>>`

Means:

T must implement `Comparable<S>`
where S is T or a supertype of T

The second form is more general.

23. Why static is important here

The method is:

```
public static <T extends Comparable<T>> T max(...)
```

The `<T ...>` belongs to the **method**, not to the class.

For example:

```
class Utility {  
    public static <T extends Comparable<T>> T max(T a, T b) {  
        return a.compareTo(b) > 0 ? a : b;  
    }  
}
```

Utility itself is not generic.

But `max()` is generic.

Therefore:

```
Utility.max(10, 20);  
Utility.max("A", "B");
```

can use different T values on different calls.

24. Static method and class-level generic

Compare:

Method-level generic

```
class Utility {  
  
    public static <T extends Comparable<T>> T max(T a, T b) {  
        return a.compareTo(b) > 0 ? a : b;  
    }  
}
```

Here T belongs to `max()`.

Class-level generic

```
class Box<T extends Comparable<T>> {  
  
    private T value;  
  
    Box(T value) {  
        this.value = value;  
    }  
}
```

Here T belongs to `Box`.

These are different scopes.

25. Complete example

```
public class GenericDemo {  
  
    public static <T extends Comparable<T>> T max(T a, T b) {  
  
        if (a.compareTo(b) > 0) {  
            return a;  
        }  
  
        return b;  
    }  
  
    public static void main(String[] args) {  
  
        Integer i = max(10, 20);  
  
        Double d = max(25.5, 12.5);  
  
        String s = max("Apple", "Banana");  
  
        System.out.println(i);  
        System.out.println(d);  
        System.out.println(s);  
    }  
}
```

Output:

```
20  
25.5  
Banana
```

26. What happens internally conceptually?

For:

```
Integer result = max(10, 20);
```

Java determines:

T = Integer

Then checks the bound:

Integer implements Comparable<Integer>

Yes.

Then the method can perform:

```
a.compareTo(b)
```

and return:

Integer

For:

```
String result = max("A", "B");
```

Java determines:

`T = String`

checks:

String implements `Comparable<String>`

Yes.

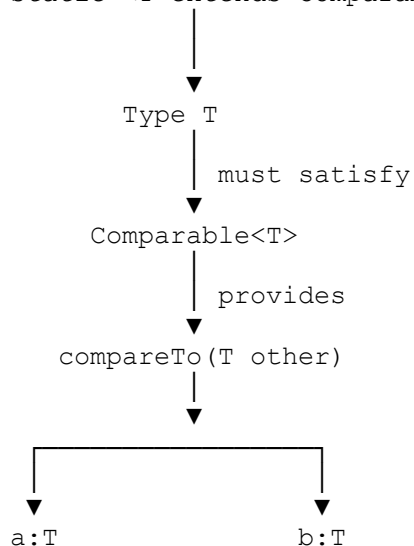
Then:

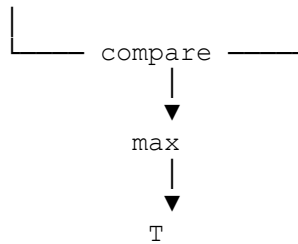
```
a.compareTo(b)
```

is valid.

27. Visual representation

```
public static <T extends Comparable<T>> T max(T a, T b)
```





28. Every component summarized

Part	Meaning
public	Method is accessible according to Java's public access rules
static	Method belongs to the class rather than an object instance
<T>	Declares a method type parameter
extends	Introduces an upper bound for T
Comparable<T>	Requires T to implement Comparable<T>
T inside Comparable<T>	Specifies what type compareTo() accepts
T before max	Return type
max	Method name
T a	First parameter
T b	Second parameter
a.compareTo(b)	Compares the two values
return a/b	Returns the larger value

29. The core concept

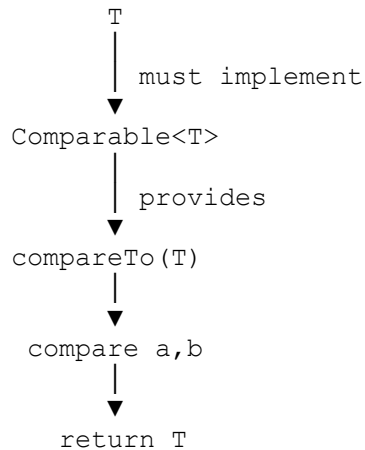
The declaration:

```
public static <T extends Comparable<T>> T max(T a, T b)
```

can be translated into plain English as:

Define a public static generic method named max. Let T be any type that implements Comparable<T>. Accept two values of that same type T, compare them using compareTo(), and return the larger value, also as type T.

The critical relationship is:



And the most important syntax to memorize is:

`<T extends Comparable<T>>`

which means:

T must be comparable to another T.

This is one of the most important concepts in Java Generics:

? extends T → safe to read as T, unsafe to write as T.

? super T → safe to write T, but you can only safely read as Object.

This is commonly remembered as **PECS**:

Producer Extends, Consumer Super

Let's understand *why*, rather than just memorizing it.

1. First understand the problem: Generic types are invariant

Suppose:

```
List<Integer> integers = new ArrayList<>();
```

You might think:

Integer → Number → Object

therefore:

List<Integer> → List<Number> → List<Object>

But Java does **not** allow that.

This is invalid:

```
List<Number> numbers = integers; // ERROR
```

Why?

Imagine Java allowed it:

```
List<Integer> integers = new ArrayList<>();
```

```
List<Number> numbers = integers; // Suppose this were allowed
```

```
numbers.add(3.14);           // Double
```

Now integers would contain:

Integer

Integer

Double ← problem!

But List<Integer> promises:

"I contain only Integer values."

Therefore Java makes generic types **invariant**:

```
List<Integer>  
  ≠  
List<Number>
```

This is the foundation for understanding wildcards.

2. What is ? extends T?

Consider:

```
List<? extends Number>
```

This means:

A List of **some unknown type** that is Number or a subclass of Number.

The unknown type could be:

```
List<Integer>  
List<Double>  
List<Long>  
List<Float>
```

but the compiler does not know exactly which one.

We can represent it as:

```
List<? extends Number>  
  |  
  ▼  
  unknown X  
  |  
  └─ X extends Number
```

The critical word is **unknown**.

3. Example of ? extends Number

```
List<Integer> integers = new ArrayList<>();
```

```
List<? extends Number> numbers = integers;
```

This is valid.

The compiler thinks:

numbers

↓

List of some unknown subtype of Number

It could secretly be:

```
List<Integer>
```

But the variable numbers doesn't expose that exact type.

4. Why is reading safe?

Suppose:

```
List<? extends Number> numbers = ...;
```

What can you safely get from it?

Whatever the hidden type is, it is guaranteed to be a Number.

For example, it could be:

Integer

Double

Long

Float

All of these are Number.

Therefore:

```
Number n = numbers.get(0);
```

is always safe.

The compiler knows:

unknown X

↓

X extends Number

Therefore:

$X \rightarrow \text{Number}$

is safe.

5. Why can't you safely write?

Consider:

```
List<Integer> integers = new ArrayList<>();  
List<? extends Number> numbers = integers;
```

Now suppose Java allowed:

```
numbers.add(10.5);
```

10.5 is a Double.

But the actual list could be:

```
List<Integer>
```

Then we would have:

```
List<Integer>
```

↑

|

Double inserted

That violates the promise of List<Integer>.

Therefore Java rejects:

```
numbers.add(10.5); // ERROR
```

It also rejects:

```
numbers.add(10); // ERROR
```

Even though 10 is an Integer.

Why?

Because the compiler doesn't know whether the hidden list is:

`List<Integer>`

or:

`List<Double>`

or:

`List<Long>`

For example:

```
List<Double> doubles = new ArrayList<>();
```

```
List<? extends Number> numbers = doubles;
```

If Java allowed:

```
numbers.add(10); // Integer
```

you'd be inserting an Integer into:

`List<Double>`

which is unsafe.

6. The compiler's perspective

When you have:

```
List<? extends Number> numbers;
```

think:

`List<X>`

where:

`X extends Number`

BUT X IS UNKNOWN

So:

`numbers.get(0)`

returns:

X

Since:

X extends Number

you can safely treat it as:

Number

But:

`numbers.add(...)`

would require you to provide an X.

You don't know what X actually is.

Therefore:

READ → X → Number SAFE
WRITE → need exact X UNSAFE

7. What can you add to `List<? extends T>`?

There is one special exception:

`numbers.add(null);`

is allowed.

Why?

Because null can be assigned to any reference type.

If the hidden type is:

Integer

null is valid.

If it is:

Double

null is valid.

If it is:

Long

null is valid.

Therefore:

```
numbers.add(null); // allowed
```

But:

```
numbers.add(10); // not allowed
```

8. Now understand ? super T

Consider:

```
List<? super Integer>
```

This means:

A List of some unknown type that is Integer or a superclass of Integer.

Possible actual types include:

```
List<Integer>
```

```
List<Number>
```

```
List<Object>
```

Conceptually:

```
List<? super Integer>
```



unknown X



X is a supertype of Integer

9. Why is writing safe?

Suppose:

List<? super Integer> list = ...;

The actual list could be:

List<Integer>

or:

List<Number>

or:

List<Object>

Now:

list.add(10);

Is 10 an Integer?

Yes.

Can an Integer be stored in:

List<Integer> → YES

List<Number> → YES

List<Object> → YES

Therefore:

list.add(10);

is always safe.

This is the key reason ? super T is a **consumer**.

10. Why is reading not safe as T?

Suppose:

List<? super Integer> list = ...;

The actual list might be:

List<Object>

Imagine:

```
List<Object> objects = new ArrayList<>();
```

```
objects.add("Hello");  
objects.add(100);
```

```
List<? super Integer> list = objects;
```

Now:

```
Integer x = list.get(0);
```

would be dangerous.

Why?

list.get(0) could return:

"Hello"

which is a String, not an Integer.

Therefore Java rejects:

```
Integer x = list.get(0); // ERROR
```

11. What CAN you read from ? super T?

You can safely read an:

Object

because every reference type ultimately extends Object.

For example:

```
Object value = list.get(0);
```

is safe.

The compiler knows:

hidden X

↓

T is a subtype of X

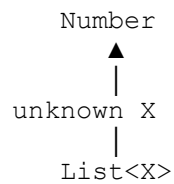
But it doesn't know that the value retrieved from the list is specifically a T.

Therefore:

READ AS Object → SAFE
READ AS T → NOT GUARANTEED

12. Visual comparison

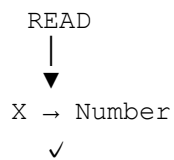
? extends T



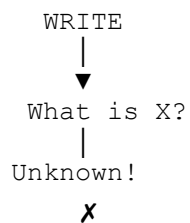
The compiler knows:

X is Number or a subclass

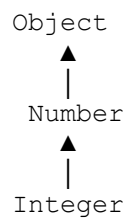
Therefore:



But writing:



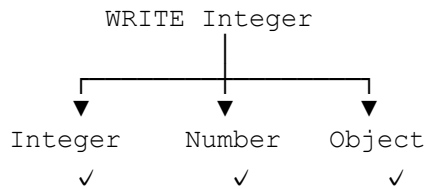
? super T



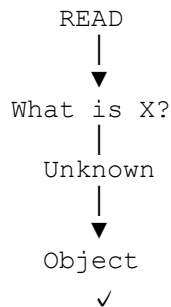
|
unknown X

More precisely, X can be Integer, Number, Object, etc.

Therefore writing:



But reading:



Only Object is guaranteed.

13. The famous PECS rule

PECS = Producer Extends, Consumer Super.

Producer → extends

If a collection **produces** values for you:

List<? extends Number>

you read from it.

Example:

```
double sum(List<? extends Number> numbers) {  
    double result = 0;  
    for (Number number : numbers) {
```

```
    result += number.doubleValue();
}

return result;
}
```

The list is a **producer** of Number values.

Therefore:

Producer → extends

Consumer → super

If a collection **consumes** values from you:

List<? super Integer>

you write to it.

Example:

```
void addNumbers(List<? super Integer> list) {

    list.add(10);
    list.add(20);
    list.add(30);
}
```

The list consumes Integer values.

Therefore:

Consumer → super

14. A practical example: copying a list

Java's Collections.copy() provides an excellent example of this principle.

Conceptually, it is:

```
static <T> void copy(
    List<? super T> destination,
    List<? extends T> source) {

    // copy values
```



```
}
```

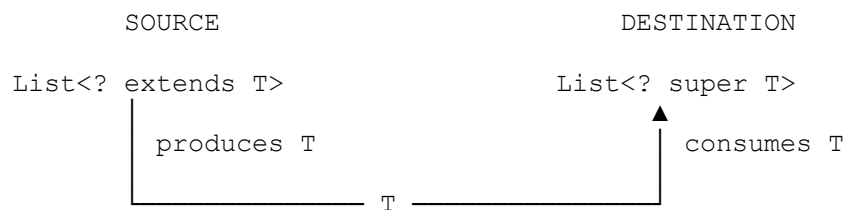
This signature is extremely powerful.

It says:

```
source  
↓  
produces T  
↓  
? extends T
```

```
destination  
↑  
consumes T  
↑  
? super T
```

Visualized:



This is exactly PECS.

15. Example of source and destination

Suppose:

```
List<Integer> source =  
    List.of(10, 20, 30);  
  
List<Number> destination =  
    new ArrayList<>(Arrays.asList(  
        0, 0, 0  
    ));
```

Then conceptually:

```
source:  
List<Integer>
```

↓
? extends Number

destination:
List<Number>

↓
? super Integer

Copying Integer values into a List<Number> is safe:

Integer
↓
Number

16. Why ? extends T is called a producer

Suppose:

List<? extends Number> numbers

You don't know whether it is:

List<Integer>
List<Double>
List<Long>

But you **can obtain** a Number:

Number n = numbers.get(0);

So the collection produces something that can be viewed as Number.

That's why:

? extends T
↓
PRODUCER

17. Why ? super T is called a consumer

Suppose:

List<? super Integer> numbers

You don't know whether it is:

List<Integer>
List<Number>
List<Object>

But you can safely give it an Integer:

```
numbers.add(10);
```

Therefore it consumes Integer.

That's why:

? super T
↓
CONSUMER

18. A common misconception

People often say:

"? super T cannot be read."

That is not completely correct.

You **can** read from it:

```
Object value = list.get(0);
```

What you cannot safely do is:

```
T value = list.get(0);
```

So the accurate statement is:

? super T is not safely readable as T; it is safely readable only as Object (or another type known to be a supertype of all possible element types).

Similarly, people sometimes say:

"? extends T cannot be written."

More precisely:

You cannot safely write a non-null value of type T into ? extends T, because the actual subtype is unknown.

null can still be added.

19. Side-by-side example

Consider:

```
List<? extends Number> producer = ...;
```

Reading

```
Number n = producer.get(0); // YES
```

Writing

```
producer.add(10);           // NO
producer.add(10.5);         // NO
producer.add(100L);         // NO
producer.add(null);         // YES
```

Now:

```
List<? super Integer> consumer = ...;
```

Writing

```
consumer.add(10);           // YES
consumer.add(20);           // YES
consumer.add(null);         // YES
```

Reading

```
Object x = consumer.get(0); // YES
```

```
Integer y = consumer.get(0); // NO
```

20. Comparison table

Operation	List<? extends T>	List<? super T>
Read as T	✓ Yes	✗ No
Read as Object	✓ Yes	✓ Yes
Add T	✗ No	✓ Yes
Add subtype of T	✗ No	✗ Generally no

Add null	✓ Yes	✓ Yes
Actual type known?	✗ No	✗ No
Acts as	Producer	Consumer
PECS	Extends	Super
Primary use	Reading	Writing

21. Why is a subtype not safe for ? super T?

Suppose:

```
List<? super Integer> list = ...;
```

You can add:

```
Integer i = 10;
list.add(i);
```

But what about:

```
Number n = 10.5;
list.add(n);
```

Not allowed.

Why?

Because the actual list could be:

```
List<Integer>
```

and a Double cannot be inserted into List<Integer>.

The fact that:

Double extends Number

doesn't help.

The guarantee is:

Integer can be stored in every possible List<? super Integer>

but:

Number cannot be stored in every possible List<? super Integer>

Therefore only Integer and its subtypes are safely writable.

22. Why isn't List<? extends Number> writable with Integer?

This is the reverse situation.

Suppose:

```
List<? extends Number> list = ...;
```

The actual list could be:

```
List<Double>
```

If you wrote:

```
list.add(10);
```

you would be putting:

```
Integer
```

into:

```
List<Double>
```

Therefore it isn't safe.

Even though:

```
Integer extends Number
```

the compiler needs to protect the **unknown specific subtype**.

23. The "unknown type" model

This is the best way to understand wildcards.

? extends T

Think:

List<? extends Number>

as:

List<X>

where:

X extends Number

but:

X = UNKNOWN

Therefore:

get() → X → Number ✓

add(Number) → need X → unknown ✗

? super T

Think:

List<? super Integer>

as:

List<X>

where:

Integer extends X

but:

X = UNKNOWN

Therefore:

add(Integer) → Integer fits X ✓

get() → X
↓

could be Object
↓
only Object guaranteed ✓

24. A complete example

```
import java.util.*;

public class WildcardDemo {

    // Producer
    static double sum(List<? extends Number> list) {

        double total = 0;

        for (Number n : list) {
            total += n.doubleValue();
        }

        return total;
    }

    // Consumer
    static void addIntegers(List<? super Integer> list) {

        list.add(10);
        list.add(20);
        list.add(30);
    }

    public static void main(String[] args) {

        List<Integer> integers =
            new ArrayList<>();

        integers.add(10);
        integers.add(20);

        double result = sum(integers);

        System.out.println(result);

        List<Number> numbers =
            new ArrayList<>();

        addIntegers(numbers);

        System.out.println(numbers);
    }
}
```



```
}  
}
```

Output:

```
30.0  
[10, 20, 30]
```

25. One method can use both

A very useful pattern is:

```
static <T> void transfer(  
    List<? extends T> source,  
    List<? super T> destination) {  
  
    for (T value : source) {  
        destination.add(value);  
    }  
}
```

Read it as:

```
source  
↓  
produces T  
↓  
? extends T
```

```
destination  
↑  
consumes T  
↑  
? super T
```

Example:

```
List<Integer> source =  
    List.of(10, 20, 30);
```

```
List<Number> destination =  
    new ArrayList<>();
```

```
transfer(source, destination);
```

This works because:

Integer $\rightarrow$ T
Integer $\rightarrow$ Number

and the destination can safely accept the Integer values.

26. The deepest reason

The rules are really about **what the compiler can guarantee**.

? extends T

The compiler knows:

Whatever the hidden type is,
it is at least a T.

Therefore:

GET $\rightarrow$ T is guaranteed
PUT $\rightarrow$ exact hidden type is unknown

Hence:

Read safely, don't write.

? super T

The compiler knows:

Whatever the hidden type is,
it is at least a supertype of T.

Therefore:

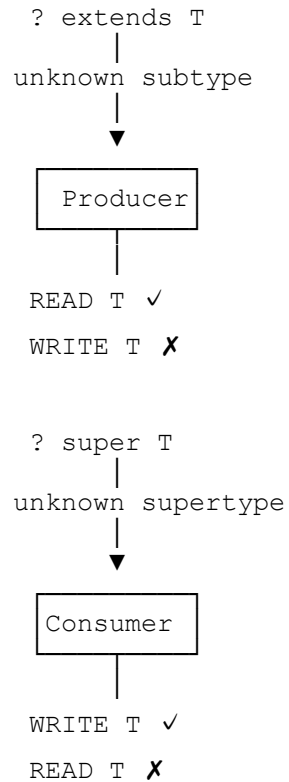
PUT T $\rightarrow$ guaranteed safe
GET $\rightarrow$ exact hidden type is unknown

Hence:

Write safely, don't read as T.

27. Final mental model

Memorize this diagram:



And the shortest possible rule is:

extends gives you values; super takes your values.

Or the classic Java mnemonic:

PECS

P → Producer

E → Extends

C → Consumer

S → Super

So:

List<? extends Number>

means:

"Give me Numbers."

while:

List<? super Integer>

means:

"I can give you Integers."

That's why ? **extends T** is safe for reading, while ? **super T** is safe for writing.